

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I Cheredla jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Answer :

```
import math
# Small dataset: [Feature1, Feature2]
X = [
    [1, 2],
    [2, 3],
    [3, 3],
    [6, 5],
    [7, 7],
    [8, 6]
]
y = ["A", "A", "A", "B", "B", "B"]
# Test data
test_data = [
    [2, 2],
    [7, 6],
    [3, 2],
    [6, 6]
]
actual = ["A", "B", "A", "B"]
# Euclidean distance
def euclidean_distance(p1, p2):
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))
# KNN prediction
def knn_predict(test_point, k):
    distances = []
    for i in range(len(X)):
        distance = euclidean_distance(test_point, X[i])
        distances.append((distance, y[i]))
    distances.sort()
    neighbors = distances[:k]
    votes = {}
    for distance, label in neighbors:
        votes[label] = votes.get(label, 0) + 1
    return max(votes, key=votes.get)
# Input K
k = int(input("Enter the value of K: "))
if k <= 0 or k > len(X):
    print("Invalid value of K")
else:
    predictions = []
```

```

print("\nPredicted Class Labels:")
for i, test_point in enumerate(test_data):
    prediction = knn_predict(test_point, k)
    predictions.append(prediction)
    print("Test instance", test_point, "->", prediction)
# Accuracy
correct = 0
for i in range(len(actual)):
    if predictions[i] == actual[i]:
        correct += 1
accuracy = (correct / len(actual)) * 100
print("\nActual Labels   :", actual)
print("Predicted Labels :", predictions)
print("Accuracy       :", accuracy, "%")

```

OUTPUT :

Enter the value of K: 3

Predicted Class Labels:

Test instance [2, 2] -> A

Test instance [7, 6] -> B

Test instance [3, 2] -> A

Test instance [6, 6] -> B

Actual Labels : ['A', 'B', 'A', 'B']

Predicted Labels : ['A', 'B', 'A', 'B']

Accuracy : 100.0 %

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Answer :

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=float)
y = np.array([2, 4, 5, 4, 5, 7, 8, 9, 10, 12], dtype=float)
# Query point
query = float(input("Enter query point: "))
# Bandwidth
tau = 1.0
# Calculate weights using Gaussian kernel
weights = np.exp(-(X - query) ** 2) / (2 * tau ** 2)
# Design matrix
X_matrix = np.vstack((np.ones(len(X)), X)).T
# Locally weighted regression
W = np.diag(weights)
theta = np.linalg.inv(
    X_matrix.T @ W @ X_matrix
) @ (X_matrix.T @ W @ y)
# Prediction using LWR
lwr_prediction = np.array([1, query]) @ theta

```

```

# Standard Linear Regression
model = LinearRegression()
model.fit(X.reshape(-1, 1), y)
linear_prediction = model.predict([[query]])[0]
# Display results
print("\nWeights:")
print(weights)
print("\nLocally Weighted Regression Prediction:",
      round(lwr_prediction, 2))
print("Standard Linear Regression Prediction:",
      round(linear_prediction, 2))
# Plot
x_plot = np.linspace(1, 10, 100)
lwr_predictions = []
for xq in x_plot:
    w = np.exp(-((X - xq) ** 2) / (2 * tau ** 2))
    W = np.diag(w)
    theta = np.linalg.inv(
        X_matrix.T @ W @ X_matrix
    ) @ (X_matrix.T @ W @ y)
    lwr_predictions.append(np.array([1, xq]) @ theta)
plt.scatter(X, y, label="Data")
plt.plot(x_plot, lwr_predictions, label="Locally Weighted Regression")
plt.plot(x_plot, model.predict(x_plot.reshape(-1, 1)),
         label="Standard Linear Regression")
plt.scatter(query, lwr_prediction, marker="x", s=100,
           label="Query Prediction")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.title("LWR vs Standard Linear Regression")
plt.show()

```

OUTPUT:

Enter query point: 6

Weights:

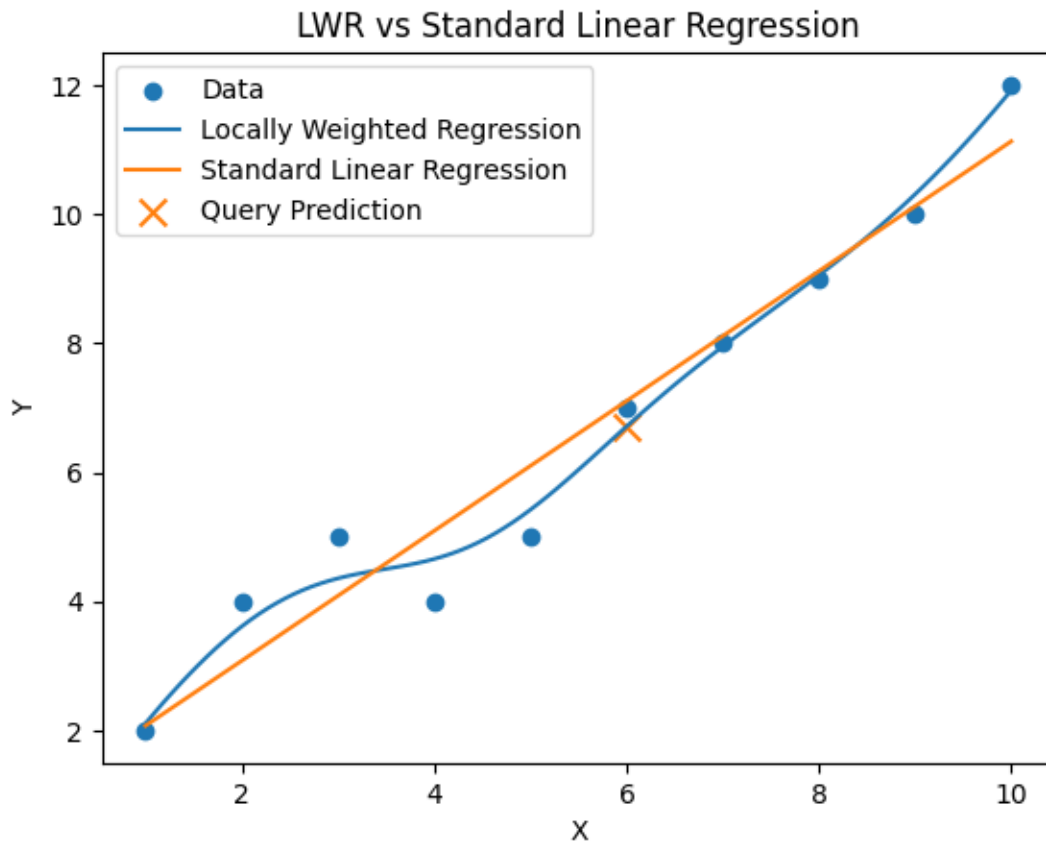
```

[3.72665317e-06 3.35462628e-04 1.11089965e-02 1.35335283e-01
 6.06530660e-01 1.00000000e+00 6.06530660e-01 1.35335283e-01
 1.11089965e-02 3.35462628e-04]

```

Locally Weighted Regression Prediction: 6.71

Standard Linear Regression Prediction: 7.1



3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Answer :

```
import numpy as np
# Small dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
y = np.array([2, 4, 5, 4, 6, 7, 8, 10], dtype=float)
# RBF centers
centers = np.array([2, 4, 6, 8], dtype=float)
# Width of Gaussian functions
width = 1.5
# Gaussian RBF function
def rbf(x, center, width):
    return np.exp(-((x - center) ** 2) / (2 * width ** 2))
# Create RBF feature matrix
R = np.zeros((len(X), len(centers)))
for i in range(len(X)):
    for j in range(len(centers)):
        R[i, j] = rbf(X[i], centers[j], width)
# Add bias
R_bias = np.column_stack((np.ones(len(X)), R))
# Train RBF network using least squares
weights = np.linalg.pinv(R_bias) @ y
```

```

# Prediction
predicted = R_bias @ weights
# Display results
print("Centers:", centers)
print("Width:", width)
print("\nPredicted Outputs:")
for i in range(len(X)):
    print("Input:", X[i],
          "Actual:", y[i],
          "Predicted:", round(predicted[i], 2))
# Mean Squared Error
mse = np.mean((y - predicted) ** 2)
print("\nMean Squared Error:", round(mse, 4))
# Test new inputs
test = np.array([2.5, 4.5, 6.5])
print("\nTest Predictions:")
for x in test:
    rbf_values = np.array([rbf(x, c, width) for c in centers])
    rbf_values = np.insert(rbf_values, 0, 1)
    prediction = rbf_values @ weights
    print("Input:", x,
          "Predicted Output:", round(prediction, 2))

```

OUTPUT:

Centers: [2. 4. 6. 8.]

Width: 1.5

Predicted Outputs:

Input: 1.0 Actual: 2.0 Predicted: 2.25
 Input: 2.0 Actual: 4.0 Predicted: 3.82
 Input: 3.0 Actual: 5.0 Predicted: 4.65
 Input: 4.0 Actual: 4.0 Predicted: 4.88
 Input: 5.0 Actual: 6.0 Predicted: 5.29
 Input: 6.0 Actual: 7.0 Predicted: 6.76
 Input: 7.0 Actual: 8.0 Predicted: 8.91
 Input: 8.0 Actual: 10.0 Predicted: 9.44

Mean Squared Error: 0.3367

Test Predictions:

Input: 2.5 Predicted Output: 4.34
 Input: 4.5 Predicted Output: 5.0
 Input: 6.5 Predicted Output: 7.86

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Answer :

```

import math
# Past cases
cases = [
    {"features": [25, 50000], "solution": "Low Risk"},
    {"features": [35, 60000], "solution": "Medium Risk"},
    {"features": [45, 80000], "solution": "High Risk"},

```

```

    {"features": [30, 55000], "solution": "Low Risk"},
    {"features": [50, 90000], "solution": "High Risk"}
]
# New query
age = float(input("Enter age: "))
salary = float(input("Enter salary: "))
query = [age, salary]
# Euclidean distance
def distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
# Find most similar case
distances = []
for case in cases:
    d = distance(query, case["features"])
    distances.append((d, case))
# Sort according to distance
distances.sort(key=lambda x: x[0])
# Closest case
closest_distance, closest_case = distances[0]
print("\nNew Query:", query)
print("Most Similar Case:", closest_case["features"])
print("Distance:", round(closest_distance, 2))
print("Predicted Solution:", closest_case["solution"])
# Display all distances
print("\nSimilarity Calculation:")
for d, case in distances:
    print("Case:", case["features"],
          "Distance:", round(d, 2))

```

OUTPUT:

Enter age: 32

Enter salary: 57000

New Query: [32.0, 57000.0]

Most Similar Case: [30, 55000]

Distance: 2000.0

Predicted Solution: Low Risk

Similarity Calculation:

Case: [30, 55000] Distance: 2000.0

Case: [35, 60000] Distance: 3000.0

Case: [25, 50000] Distance: 7000.0

Case: [45, 80000] Distance: 23000.0

Case: [50, 90000] Distance: 33000.0

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

Answer :

```

import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

```

```

# Dataset
X = np.array([
    [1, 1],
    [2, 2],
    [2, 3],
    [3, 3],
    [6, 6],
    [7, 7],
    [8, 7],
    [8, 8]
])
y = np.array([
    0, 0, 0, 0,
    1, 1, 1, 1
])
# Test data
X_test = np.array([
    [2, 2],
    [3, 2],
    [7, 6],
    [8, 8]
])
y_test = np.array([0, 0, 1, 1])
# ----- KNN MODEL -----
k = 3
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X, y)
knn_predictions = knn.predict(X_test)
# ----- RBF MODEL -----
# RBF centers
centers = np.array([
    [1.5, 1.5],
    [2.5, 2.5],
    [6.5, 6.5],
    [7.5, 7.5]
])
# Gaussian width
width = 1.5
# Gaussian RBF function
def gaussian_rbf(X, centers, width):
    result = []
    for point in X:
        values = []
        for center in centers:
            distance = np.linalg.norm(point - center)
            value = np.exp(-(distance ** 2) / (2 * width ** 2))
            values.append(value)
        result.append(values)
    return np.array(result)

```



```

# Generate RBF features
R = gaussian_rbf(X, centers, width)
# Add bias
R_bias = np.column_stack((np.ones(len(R)), R))
# Train RBF weights
weights = np.linalg.pinv(R_bias) @ y
# Test RBF features
R_test = gaussian_rbf(X_test, centers, width)
R_test_bias = np.column_stack((np.ones(len(R_test)), R_test))
# RBF output
rbf_output = R_test_bias @ weights
# Convert output to class
rbf_predictions = (rbf_output >= 0.5).astype(int)
# ----- ACCURACY -----
knn_accuracy = accuracy_score(y_test, knn_predictions)
rbf_accuracy = accuracy_score(y_test, rbf_predictions)
# ----- DISPLAY RESULTS -----
print("KNN vs RBF COMPARISON")
print("-" * 50)
print("\nTest Input\tActual\tKNN\tRBF Output\tRBF Class")
for i in range(len(X_test)):
    print(
        X_test[i],
        "\t",
        y_test[i],
        "\t",
        knn_predictions[i],
        "\t",
        round(rbf_output[i], 3),
        "\t\t",
        rbf_predictions[i]
    )
print("\nAccuracy:")
print("KNN Accuracy :", round(knn_accuracy * 100, 2), "%")
print("RBF Accuracy :", round(rbf_accuracy * 100, 2), "%")
print("\nAnalysis:")
print("1. KNN uses distance to find the K nearest training samples.")
print("2. RBF uses Gaussian functions to transform the input.")
print("3. KNN does not require training weights.")
print("4. RBF learns weights from the transformed features.")
print("5. KNN gives direct class labels.")
print("6. RBF produces continuous outputs before converting them to classes.")

```

OUTPUT :

KNN vs RBF COMPARISON

Test Input	Actual	KNN	RBF Output	RBF Class
[2 2] 0	0	-0.106		0
[3 2] 0	0	0.003	0	
[7 6] 1	1	0.997	1	
[8 8] 1	1	0.928	1	

Accuracy:**KNN Accuracy : 100.0 %****RBF Accuracy : 100.0 %****Analysis:**

1. KNN uses distance to find the K nearest training samples.
2. RBF uses Gaussian functions to transform the input.
3. KNN does not require training weights.
4. RBF learns weights from the transformed features.
5. KNN gives direct class labels.
6. RBF produces continuous outputs before converting them to classes.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand